Deep Learning for Natural Language Processing

Today's goal • 3/3 items

Module 1
Feedforward Neural Nets and Recurrent Neural Networks

Module 2
Sequence to Sequence Models, Attention, Transformers

Module 3
Transfer Learning

Module 4
Large Language Models

Large Language Models and How to Use Them

- Large Language Models and Emergent Abilities
  Video • 11 min
- Parameter-Efficient Finetuning
  Video • 6 min
- Prompting, Prompt Engineering, and In-Context Learning
  Video • 10 min
- Prompting Challenges
  Video • 7 min
- Large Language Models and How to Use Them
  Practice Assignment • Grade: 100%

Language-and-Vision Models

- The Limits of Language-only Models
  Video • 6 min
- Popular Pretrained Language-and-Vision Models
  Video • 7 min
- Multimodality Beyond Vision
  Video • 7 min
- Language-and-Vision Models
  Practice Assignment • Grade: 100%

Non-functional Properties of Large Language Models

- Hallucinations
  Video • 10 min
- Fairness and Model Alignment
  Video • 8 min
- Resource Efficiency and Knowledge Distillation
  Video • 8 min
- Privacy
  Video • 7 min
- Interpretability
  Video • 9 min
- Non-functional Properties of Large Language Models
  Practice Assignment • Grade: 100%

Review and Practice

- Large Language Models
  Graded Assignment • Grade: 100%
- **Large Language Models**
  Programming Assignment • 2h

Programming assignment

Large Language Models

Launch lab



Instructions | My submissions | Discussions

Large Language Models (LLMs) like GPT-4, Claude, and LLaMA have revolutionized natural language processing. While these models have billions of parameters and are trained on vast datasets, they share fundamental architectural similarities with the model you'll build in this assignment. Most modern LLMs are based on decoder-only transformer architectures that generate text auto-regressively.

Auto-regressive language modeling means the model predicts the next token in a sequence based on all previous tokens. This approach allows the model to generate coherent text by repeatedly predicting and sampling new tokens, conditioned on the tokens it has already generated. The "language modeling" objective—predicting the next word given previous words—allows these models to learn syntax, semantics, factual knowledge, and even reasoning capabilities, all from the patterns in text.

In this assignment, you'll implement a word-level language model using the tiny Shakespeare dataset, which contains the complete works of William Shakespeare (~1MB of text). While small by modern standards, this dataset provides enough structure and patterns for our model to learn meaningful representations and generate Shakespeare-like text. The language, featuring archaic words and poetic structure, presents an interesting stylistic target for our model. Through building this transformer-based architecture and training it to predict the next word in Shakespeare's texts, you'll gain insight into the core mechanisms that power today's most advanced language models—just at a much smaller scale.

In this assignment, you will build and train a word-level language model using PyTorch. You'll start by preprocessing text data, constructing a vocabulary, and encoding sentences into numerical representations. Using this processed data, you will implement key components of a transformer-based language model, including positional encoding, self-attention mechanisms, and feed-forward networks.

By the end, you will have hands-on experience with building and training neural language models, helping you understand how modern language models process and generate text.

Report an issue

What to expect

Due Nov 23, 11:59 PM IST

Unlimited attempts

Go to My Learning